BÁO CÁO PHÂN CÔNG & THIẾT KẾ TRIỂN KHAI

QUERY UNDERSTANDING & ROUTING | LONG – TRÍ

Bản hợp nhất yêu cầu dự án, bản phân công ngày 22/10/2026 và các góp ý bổ sung từ ảnh.

Trạng thái: tài liệu đề xuất triển khai; các thay đổi schema cần được Long, Trí và nhóm Retrieval thống nhất trước khi khóa contract.

1. Tổng quan và phạm vi

Module là cầu nối từ câu hỏi người dùng tới Retrieval (Kiệt/Diệp). Nhiệm vụ chính: kiểm tra input, chuẩn hóa, dùng LLM hiểu intent/entities/filter/tasks, xử lý câu hỏi mơ hồ và điều hướng kết quả thành query_plan nhất quán.

Luồng xử lý thống nhất: Raw query → Validation + Safety Screening (Trí) → Normalization (Trí) → Understanding + Clarification (Long) → Routing (Trí) → Retrieval (Kiệt/Diệp). Trường hợp không hợp lệ / nguy hiểm / mơ hồ / ngoài phạm vi được chặn hoặc hỏi lại trước Retrieval.

Người	Phạm vi phụ trách	Đầu ra chính
Long	Ontology YAML; lựa chọn và prompt LLM; intent, entities, filters, multi-task; conversation context; clarification	ontology.yaml, understanding_module.py
Trí	Validation; safety screening & policy routing; normalization; alias mapping; routing/glue code; integration	validation_module.py, normalization_module.py, routing_module.py
Cả hai	Khóa interface/schema; kiểm thử tích hợp và bàn giao cho Retrieval	query_plan contract, test cases

2. Phần việc của Long

2.1 Ontology YAML cho 5 lỗ hổng

Xây dựng ontology.yaml dùng chung cho XSS, SQL Injection, Broken Access Control, Authentication Failures, Security Misconfiguration; mỗi entry gồm canonical, aliases, cwe, subtypes, related_terms. Field không có dữ liệu là [] thay vì bỏ field. Chuẩn slug dùng chung: xss, sqli, bac, auth_failures, misconfig. Cần đối chiếu OWASP/MITRE, rà lại alias để không nhầm related_terms thành từ đồng nghĩa.

```
xss:
  canonical: "Cross-Site Scripting"
  aliases: ["xss", "cross-site scripting", "cross site scripting"]
  cwe: ["CWE-79"]
  subtypes: ["stored", "reflected", "dom-based"]
  related_terms: ["csp", "output encoding", "sanitization", "escaping"]
```

Giao file cho Trí để bổ sung alias mapping; Trí chỉ đọc, đề xuất sửa qua Long. Quan trọng: bước normalize cơ bản của Trí KHÔNG cần chờ YAML.

2.2 Query Understanding bằng LLM

- Đánh giá các LLM chạy local theo tài nguyên thực tế; danh sách thử nghiệm trong bản gốc: Qwen3 → meta-llama → deepseek-ai. Đánh giá độ đúng intent, entity extraction, tốc độ và tính ổn định JSON.
- Classify một intent tổng thể: SEARCH, COMPARE, SYNTHESIS, CLARIFICATION hoặc OUT_OF_SCOPE.
- Extract entities.document[] và entities.vulnerabilities[] (chuẩn slug theo ontology), filter.version, filter.document_type, filter.source[].
- Tách nhiều yêu cầu thành task[] với task_id, intent_task, query, objective, depends_on; trả confidence dạng float 0–1.

2.3 Bổ sung: Quy tắc xác định context_used (từ ảnh)

Long phải thiết kế rõ điều kiện dùng lịch sử hội thoại; không được gán context_used tùy ý. Trường này là true CHỈ KHI câu hỏi hiện tại được giải nghĩa, mở rộng hoặc khử tham chiếu nhờ nội dung lịch sử hội thoại được đưa vào prompt; nếu hiểu câu hỏi độc lập thì false. Không đánh dấu true đơn thuần vì prompt có lịch sử nhưng không dùng nó.

- Ví dụ: trước đó hỏi “XSS là gì?”, sau đó hỏi “Còn cách phòng tránh nó?” → context_used=true; resolved_query có thể làm rõ “nó” = XSS.
- Ví dụ: “So sánh SQLi và XSS” là câu độc lập → context_used=false dù có history.
- Thiết kế prompt nêu rõ chỉ dùng lịch sử tin cậy để giải quyết tham chiếu; không suy đoán thông tin thiếu. Nếu lịch sử không đủ rõ, kích hoạt clarification.
- Chỉ gửi phần lịch sử cần thiết, có giới hạn độ dài và thứ tự; nội dung từ history là dữ liệu, không được ghi đè chỉ dẫn hệ thống.

2.4 Clarification logic

Long thiết lập threshold cho confidence và bộ câu hỏi clarification cụ thể. Khi need_clarify=true, clarification_message bắt buộc khác null; không gửi sang Retrieval. Threshold là tham số cần hiệu chỉnh bằng tập test, không coi confidence do LLM tự khai báo là xác suất đã hiệu chuẩn.

3. Phần việc của Trí

3.1 Query Validation — kiểm tra cấu trúc và spam

- Kiểm tra query rỗng / toàn khoảng trắng; độ dài hợp lý (gợi ý ban đầu 3–500 ký tự, cho phép cấu hình); spam, chuỗi lặp bất thường, ký tự điều khiển không hợp lệ.
- Áp dụng Regex/rule phát hiện tín hiệu bất thường, nhưng tránh từ chối máy móc chỉ vì một câu hỏi học thuật chứa “SQL Injection”, mã SQL, hoặc <script>: hệ thống vốn phục vụ nội dung bảo mật.
- Trả {"is_valid": true, "reject_reason": null} khi qua bước kiểm tra; thất bại trả is_valid=false cùng mã reject_reason rõ ràng.

3.2 Bổ sung: Safety Screening & Safety Routing (từ ảnh)

Bổ sung bước đánh giá MỤC ĐÍCH của truy vấn, không chỉ dò keyword/Regex. Phân biệt yêu cầu học tập, phân tích, phòng vệ, kiểm thử có ủy quyền với yêu cầu hướng dẫn tấn công gây hại. Ví dụ minh họa trong ảnh: “Cho tôi hướng dẫn hack trang web này” cần được chặn/đưa sang phản hồi an toàn thay vì chuyển thành truy vấn Retrieval hướng dẫn khai thác.

Thiết kế sơ bộ (đề xuất kỹ thuật): lớp regex/rule nhanh → phân loại safety theo ngữ cảnh nếu tín hiệu đáng ngờ → quyết định ALLOW / REVIEW_OR_CLARIFY / REFUSE. Với trường hợp không chắc chắn, ưu tiên hỏi phạm vi/ủy quyền và mục tiêu phòng vệ thay vì kết luận từ một từ khóa. Có thể dùng logic rule-based trước và thêm classifier ở giai đoạn nâng cấp.

- ALLOW: câu hỏi hợp lệ về kiến thức, kiểm thử trong môi trường được phép, phòng ngừa, khắc phục → tiếp tục Understanding/Retrieval.
- REVIEW_OR_CLARIFY: chưa rõ mục đích/phạm vi, cần hỏi lại; chưa cấp hướng dẫn khai thác.
- REFUSE: yêu cầu rõ ràng hỗ trợ xâm nhập hay gây hại → dừng pipeline, trả thông báo ngắn gọn và chuyển hướng sang bảo mật phòng thủ.

Mở rộng output khi từ chối: có reject_reason là mã máy đọc được, đồng thời reason là diễn giải ngắn cho người dùng; không tiết lộ rule phát hiện nhạy cảm. Đây là yêu cầu mới từ ảnh, cần cập nhật contract đầu ra.

```
{
  "is_valid": false,
  "reject_reason": "unsafe_attack_intent",
  "reason": "Không thể hỗ trợ xâm nhập trái phép; có thể hỗ trợ kiểm tra an toàn trong phạm vi được phép.",
  "safety_action": "REFUSE"
}
```

Các mã reject_reason thống nhất đề xuất: empty_query, query_too_short, query_too_long, spam_detected, suspicious_pattern, unsafe_attack_intent. Với query bị gắn cờ nhưng chưa kết luận nguy hiểm, sử dụng safety_action=REVIEW_OR_CLARIFY thay vì false-positive refusal.

3.3 Query Normalization

- Pha A – làm độc lập ngay: lowercase cho working text, trim đầu/cuối, chuẩn hóa nhiều khoảng trắng và các ký tự lỗi đã xác định; giữ nguyên nội dung code/payload nếu cần để không làm sai ý nghĩa.
- Pha B – sau khi Long chia sẻ ontology.yaml: map aliases → canonical/slugs theo từ điển; không thay thế substring mù quáng làm biến dạng query.
- original_query lưu nguyên từng ký tự user nhập; resolved_query là bản đã chuẩn hóa dùng cho LLM. Quy ước riêng với snippet code và trích dẫn cần được test.

3.4 Routing / Glue Code

Trí ghép kết quả Validation/Safety và Understanding. Thứ tự ưu tiên đề xuất: invalid input → safety REFUSE → safety REVIEW_OR_CLARIFY → need_clarify / CLARIFICATION → OUT_OF_SCOPE → SEARCH/COMPARE/SYNTHESIS sang Retrieval. Khóa thứ tự nhánh này cùng Long để tránh kết quả mâu thuẫn.

```
def route(validation_result: dict, understanding_result: dict) -> dict:
  # Kiểm tra trạng thái và schema; trả reject/clarification
  # hoặc query_plan đã validate. Phần hiện thực do Trí đảm nhiệm.
  ...
```

4. Interface Long – Trí và các ràng buộc

Hướng kết nối	Kiểu dữ liệu	Yêu cầu
Trí → Long	resolved_query: str	Không mất original_query; có thể kèm history qua interface đã thống nhất
Long → Trí	understanding_result: dict	intent, confidence, context_used, entities, filter, task, clarification
Long → Trí	ontology.yaml	5 slug; canonical / aliases / subtypes / cwe / related_terms
Trí → Retrieval	query_plan: dict	Chỉ khi valid, an toàn, đủ rõ và trong phạm vi

```
# Long
```

```
understand_query(resolved_query: str, conversation_history: list | None = None) -> dict
# Trí
validate_query(raw_query: str) -> dict
normalize_query(raw_query: str, ontology: dict | None = None) -> str
route(validation_result: dict, understanding_result: dict) -> dict
```

Lưu ý interface: conversation_history=None tránh mutable default list. Đây là chỉnh sửa kỹ thuật được đề xuất so với signature list=[] trong bản phân công, không phải yêu cầu gốc.

5. Output Schema chung — bản đề xuất cập nhật

Giữ lại các field gốc của query_plan; bổ sung reason/safety_action cho nhánh từ chối, và chuẩn hóa task.objective thành đối tượng có cấu trúc thay vì chuỗi tùy ý. Cách này loại bỏ bất nhất giữa "vulnerabilities: [xss, sql_injection]" và "vulnerabilities:xss". Thay đổi objective cần nhóm Retrieval phê duyệt vì ảnh hưởng contract.

```
{
 "original_query": "So sánh XSS và SQLi, đồng thời tìm cách phòng chống XSS",
 "resolved_query": "so sánh Cross-Site Scripting và SQL Injection, đồng thời tìm cách phòng chống Cross-Site Scripting",
 "intent": "COMPARE",
 "confidence": 0.92,
 "context_used": false,
 "entities": {"document": [], "vulnerabilities": ["xss", "sqli"]},
 "filter": {"version": null, "document_type": null, "source": []},
 "task": [
  {"task_id": "task_001", "intent_task": "COMPARE",
   "query": "so sánh XSS và SQL Injection",
   "objective": {"type": "vulnerabilities", "ids": ["xss", "sqli"]},
   "depends_on": null},
  {"task_id": "task_002", "intent_task": "SEARCH",
   "query": "cách phòng chống XSS",
   "objective": {"type": "vulnerabilities", "ids": ["xss"]},
   "depends_on": null}
 ],
 "need_clarify": false,
 "clarification_message": null
}
```

Quy tắc objective: object {"type": string, "ids": string[]} với type thuộc tập được cả nhóm thống nhất, ví dụ vulnerabilities, documents. Nếu một task cần nhiều nhóm mục tiêu, cân nhắc mở rộng thành objectives[] ở phiên bản schema sau, không tự chuyển kiểu giữa các lần chạy. depends_on là null hoặc task_id tồn tại; task[] luôn là mảng.

Rejection response là một nhánh output KHÁC query_plan: {"status":"rejected", "reject_reason":"unsafe_attack_intent", "reason":"...", "safety_action":"REFUSE"}. Đề xuất thêm status rõ ràng cho các nhánh khác như clarification / out_of_scope để tích hợp UI dễ hơn; các trường này chỉ là đề xuất bổ sung, không có trong schema gốc.

6. Trình tự ưu tiên và phụ thuộc

Thứ tự	Người	Việc cần hoàn thành	Phụ thuộc
1	Trí	Validation cấu trúc, quy tắc spam + bộ normalize cơ bản	Không

1 (song song)	Long	Ontology YAML với 5 lỗ hổng và slug cố định	Không
2	Trí	Safety screening & reason/reject_reason; test false positives	Validation nền
2 (song song)	Long	Chọn/test local LLM; viết prompt Understanding + context_used	Có thể dùng query mẫu
3	Long → Trí	Share ontology.yaml; thống nhất schema/objective	Ontology xong
4	Trí	Alias mapping theo YAML	ontology.yaml
4 (song song)	Long	Clarification threshold + multi-task parsing	LLM hiểu query
5	Trí	Routing + schema validation + mock Understanding	Contract thống nhất
6	Cả hai	Integration test end-to-end; bàn giao Kiệt/Diệp	Các module xong

Khác bản cũ: Trí không phải chờ ontology.yaml để bắt đầu normalize. Chỉ riêng alias mapping phụ thuộc Long; phần rule cơ bản triển khai song song được.

7. Checklist nghiệm thu

- ☐ Validation chặn query rỗng/quá dài/spam, trả reject_reason xác định; không gọi LLM khi chặn.
- ☐ Safety phân biệt truy vấn bảo mật phòng vệ với ý định gây hại; không chặn nhầm chỉ vì chứa cú pháp SQL/script.
- ☐ Nhánh refusal có reason dễ hiểu và điều hướng sang nội dung phòng thủ; nhánh chưa rõ có thể hỏi làm rõ.
- ☐ Normalization giữ original_query 100%; pha cơ bản chạy không cần YAML; alias mapping theo đúng slug.
- ☐ LLM luôn trả intent trong 5 nhãn, confidence 0–1; JSON hợp lệ và field entities/filter luôn có.
- ☐ context_used chỉ true khi lịch sử thực sự hỗ trợ hiểu câu hiện tại; trường hợp tham chiếu mơ hồ được kiểm thử.
- ☐ Clarification có message cụ thể nếu need_clarify=true; không chuyển Retrieval.
- ☐ Multi-task xuất task[] nhất quán; objective dùng cùng một kiểu; depends_on chỉ tham chiếu task hợp lệ.
- ☐ Routing đầy đủ nhánh reject / review-clarify / out-of-scope / retrieval; không gọi downstream sai.
- ☐ Kiệt/Diệp xác nhận các thay đổi schema trước khi tích hợp; ghi rõ version của contract.

8. Ma trận test tối thiểu

Input minh họa	Kỳ vọng chính
" "	Reject: empty_query
"XSS là gì?"	ALLOW → SEARCH, vulnerabilities=[xss]
"Có thể phòng chống SQL Injection ra sao?"	ALLOW → SEARCH
"<script>alert(1)</script> trong ví dụ XSS là gì?"	Không reject tự động theo keyword; xem xét ngữ cảnh học thuật
"Hướng dẫn hack một website không được	Safety REFUSE + reason + hướng phòng thủ

phép"	
"Cách khai thác lỗi này?" (không có context rõ)	REVIEW_OR_CLARIFY / clarification
"Còn cách vá nó?" sau chủ đề XSS	context_used=true, tham chiếu tới XSS nếu lịch sử đủ rõ
"So sánh XSS và SQLi, rồi tóm tắt cách phòng tránh"	task[] >= 2; objective đúng kiểu
"Tôi muốn biết thời tiết"	OUT_OF_SCOPE
"Các lỗ hổng nguy hiểm"	Có thể need_clarify=true với câu hỏi cụ thể

9. Danh mục bàn giao và điểm cần chốt

- Long: ontology.yaml; understanding_module.py; prompt/config LLM; bộ test intent/context/clarification.
- Trí: validation_module.py; normalization_module.py; routing_module.py; bộ test Safety Routing và luồng điều hướng.
- Chung: schema query_plan phiên bản cố định; hợp đồng objective; thống nhất error/rejection envelope; test end-to-end; mô tả interface cho Retrieval.

Ba điểm cần khóa trước tích hợp: (1) danh sách mã safety_action/reject_reason; (2) cấu trúc objective là object thay vì string; (3) format nhánh reject/clarify và cách truyền conversation_history. Đây là các đề xuất bổ sung để giải quyết góp ý trong ảnh, không được tự coi là đã thống nhất trong nhóm.

10. Cơ sở tài liệu

Nguồn 1: Note-Computational-Thinking (5).pdf, mục “Query Understanding and Routing => Long, Trí”, trang 11–12. Nguồn 2: “Phân chia công việc Query Understanding & Routing(1).pdf”, 12 trang, ngày 22/10/2026. Nguồn 3: Ảnh góp ý image(4).png — bổ sung Safety Routing, reason, context_used, objective và thứ tự ưu tiên. Các biện pháp kỹ thuật được diễn giải thêm trong báo cáo được đánh dấu là “đề xuất”.